

บทที่ 4

เอชทีทีพี รูปแบบเว็บ เฟตช์เอพีไอ และการวินิจฉัยเครือข่าย

บทนำ

เมื่อเบราว์เซอร์และเซิร์ฟเวอร์ทำงานคนละกระบวนการ ทั้งสองฝ่ายต้องสื่อสาร ผ่านเกณฑ์วิธี เอชทีทีพี (Hypertext Transfer Protocol: HTTP) กำหนดความหมายของคำขอและการตอบกลับ ส่วนเจสัน (JavaScript Object Notation: JSON) เป็นรูปแบบข้อมูลที่นิยมใช้ ความเข้าใจเกี่ยวกับวิธีการ เป้าหมาย ส่วนหัว รหัสสถานะ และส่วนเนื้อหา ช่วยให้พัฒนาแก้ปัญหาเครือข่ายออกจากปัญหา ในส่วนติดต่อผู้ใช้และฐานข้อมูลได้อย่างมีประสิทธิภาพ

เอกสารอ้างอิงหัวข้อ 5-1 ถึง 6-4 เพิ่มภาพรวมที่สำคัญอีกสองเรื่อง เรื่องแรกคือเว็บมีหลายวิธีสื่อสาร เช่น HTTP แบบคำขอและคำตอบ การสำรวจข้อมูลเป็นช่วงเวลา (polling) เหตุการณ์ที่เซิร์ฟเวอร์ส่งมา (Server-Sent Events: SSE) และเว็บซ็อกเก็ต (WebSocket) เรื่องที่สองคือข้อมูลบนเว็บมีหลายรูปแบบ เช่น CSV, XML, JSON, รูปภาพ, วิดีโอ และเสียง บทนี้จึงอ่านทั้ง **วิธีส่ง** และ **รูปแบบที่ส่ง** ผ่านกรณีระบบคิวร้านกาแฟ

ภาพหน้าจอในรูปที่ 4.1 เป็นเอกสารของ MDN สำหรับ Fetch API ขณะอ่าน ให้หาคำประกอบสามส่วนของตัวอย่างได้แก่ ที่อยู่ปลายทาง วิธีส่งคำขอ และจุดที่โค้ดอ่านการตอบกลับ แล้วนำสามส่วนนี้มาเทียบกับคำสั่งซื้อของร้านกาแฟ

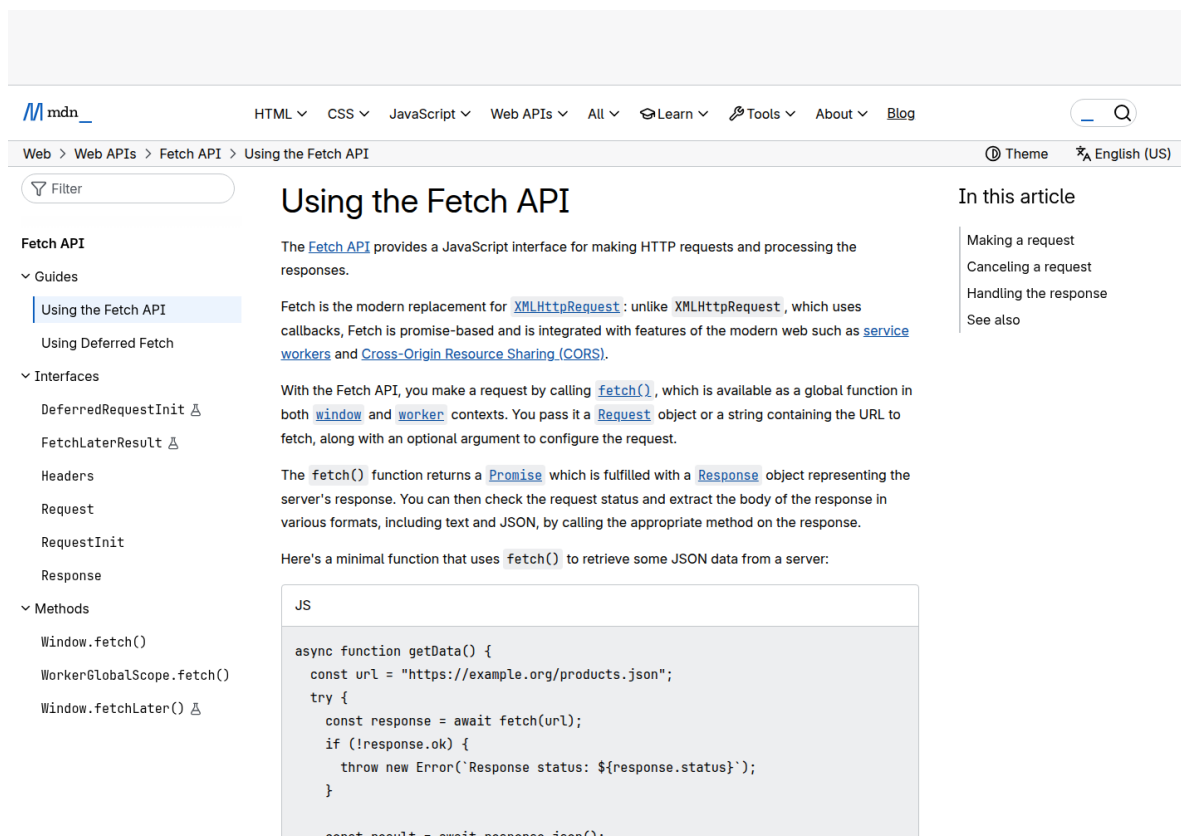
4.1 ข้อตกลงระหว่างโปรแกรม

HTTP เปรียบได้กับข้อความบนใบสั่งกาแฟที่ส่งจากหน้าร้านไปหลังร้าน ข้อความต้องบอกว่า เราขอทำอะไรกับสิ่งใด และแนบข้อมูลที่จำเป็น แคลเอ็นด์ตอบด้วยผลลัพธ์และสถานะ ที่มีความหมายร่วมกัน จึงไม่ควรให้พรอนต์เอ็นด์เดาโครงสร้างคำตอบจากการทดลองหลายครั้ง ข้อตกลงของ API คือความรู้ร่วมที่ทำให้สองโปรแกรมพัฒนาแยกกันได้อย่างยังทำงานร่วมกันได้

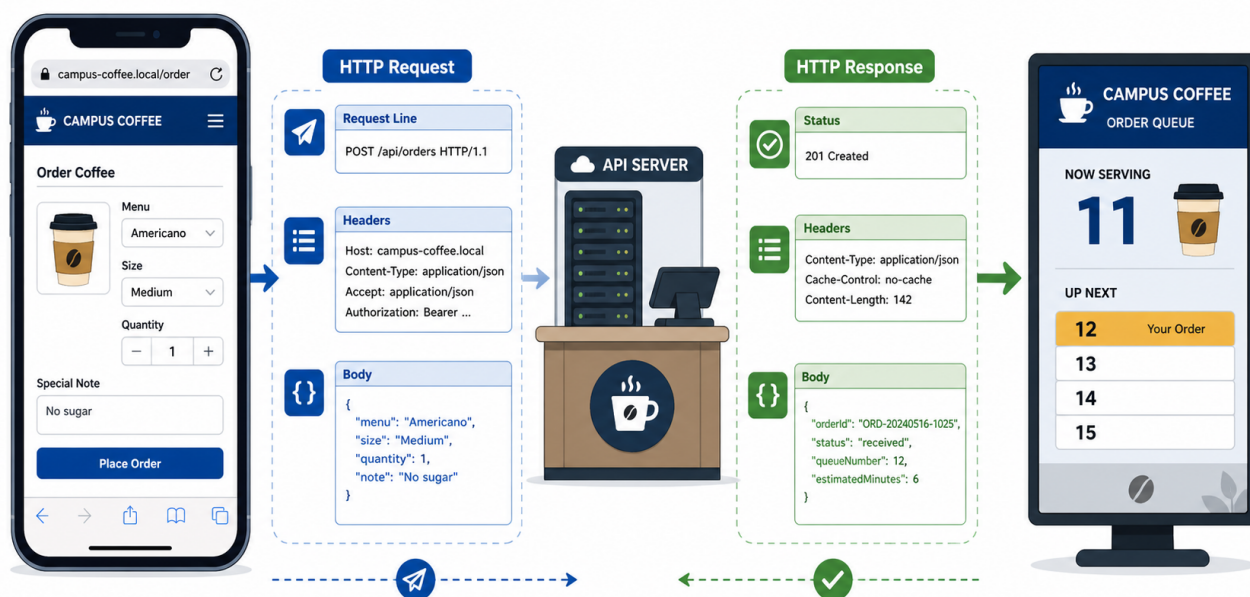
JSON เป็นรูปแบบแทนข้อมูล ความหมายของข้อมูลเกิดจากชื่อเขตข้อมูล ชนิดข้อมูล กรณีที่ยอมรับได้ และผลเมื่อเกิดกาไม่ผ่าน บทนี้แถมจะใช้ข้อตกลงนี้จัดสถานะของหน้า ส่วนบทแคลเอ็นด์จะทำให้การตรวจและการตอบกลับมีเจ้าของที่ชัดเจน

4.2 ผลลัพธ์การเรียนรู้ประจำบท

1. แยกองค์ประกอบของคำขอและการตอบกลับ HTTP ได้
2. เลือกวิธีการ (method) และรหัสสถานะ (status code) ตามความหมาย โดยอธิบายผลต่อทรัพยากรได้
3. ใช้ส่วนหัว (header), MIME type และคุกกี้ (cookie) เพื่ออธิบายข้อมูลกำกับของการสื่อสารได้
4. ออกแบบข้อตกลงข้อมูล JSON สำหรับกรณีสำเร็จและกรณีผิดพลาด



รูปที่ 4.1: ภาพหน้าจอเอกสาร MDN เรื่อง Fetch API: ใช้สำหรับการสร้างคำขอและการรับคำตอบ



รูปที่ 4.2: คำสั่งซื้อหนึ่งคำสั่งเดินทางเป็น HTTP request แล้วกลับมาเป็น HTTP response ที่หน้าคิว

5. เลือกรูปแบบข้อมูล รูปภาพ วิดีโอ และเสียงให้เหมาะกับงานของหน้าเว็บ
6. เปรียบเทียบ polling, long polling, SSE และ WebSocket ตามจังหวะของข้อมูล
7. ใช้เฟตช์เอพีไอ (Fetch API) พร้อมแยกข้อผิดพลาดของเครือข่าย ออกจากข้อผิดพลาดตามสถานะ HTTP
8. ใช้ `curl` และแพนเครือข่าย (Network) ทำให้ปัญหาเกิดซ้ำได้

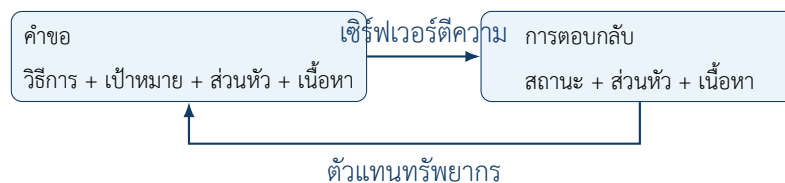
เส้นทางของเรา: ข้อตกลงระหว่างฟรอนต์เอนด์และแบคเอนด์

เมื่อจบบทนี้ เราควรอธิบายได้ว่าการกด “สั่งลาเต้เย็น” กลายเป็นคำขอ HTTP อย่างไร และคำตอบชนิดใดต้องทำให้หน้าจอแสดงคิวนัด ข้อผิดพลาด หรือการลองใหม่ สิ่งที่ต้องทำคือกำหนด endpoint สำหรับเมนู คำสั่งซื้อ และสถานะคิวนัด พร้อมตัวอย่าง JSON ทั้งสำเร็จและล้มเหลว ผลลัพธ์คือข้อตกลง API ที่ทีมฟรอนต์เอนด์และทีมแบคเอนด์ใช้ร่วมกันได้

กรณีใกล้ตัวล่าสุด: PromptPay และ QR Payment ทำให้ HTTP failure เป็นเรื่องใกล้ตัว

ร้านกาแฟในมหาวิทยาลัยมักรับเงินผ่าน QR หรือแอปธนาคาร Bangkok Bank เคยประกาศปิดปรับปรุงบริการในวันที่ 8 พฤศจิกายน 2568 ซึ่งกระทบ Mobile Banking, PromptPay, QR Payment และ API Payment [35] ส่วน TTB ประกาศปิดระบบ PromptPay คืบวันที่ 11 มิถุนายน 2569 ถึงเช้าวันที่ 12 มิถุนายน 2569 ทำให้ลูกค้าไม่สามารถโอน รับโอน หรือรับชำระ QR ได้ในช่วงนั้น [36] เราจึงต้องออกแบบหน้าร้านให้บอกสถานะ และให้ทางเลือก เช่น ชำระเงินที่เคาน์เตอร์หรือบันทึกคำสั่งซื้อไว้รอชำระ

4.3 แบบจำลองข้อความ HTTP



รูปที่ 4.3: องค์ประกอบเชิงความหมายของการแลกเปลี่ยน HTTP

ความหมายเชิงมาตรฐานของ HTTP กำหนดความหมายร่วมของวิธีการ รหัสสถานะ และช่องข้อมูลส่วนหัว โดยไม่ผูกกับหน้าตาของแอปพลิเคชัน[15] ตัวอย่างคำขอมีดังนี้

```
POST /api/orders HTTP/1.1
Host: localhost:3000
Content-Type: application/json
Accept: application/json

{"drink": "Iced Latte", "sweetness": 50, "customerName": "Mali"}
```

บรรทัดแรกกำหนดวิธีการและเป้าหมาย ส่วนหัวอธิบายข้อมูลกำกับ บรรทัดว่างแยกส่วนหัวออกจากส่วนเนื้อหา การอ่านบันทึกเหตุการณ์เฉพาะเนื้อหา จึงไม่เพียงพอ เพราะปัญหาอาจอยู่ที่วิธีการ เส้นทาง หรือชนิดสื่อ

4.4 ส่วนหัว MIME type และคุกกี้

ส่วนหัว HTTP เป็นข้อมูลกำกับที่ช่วยให้ไคลเอนต์และเซิร์ฟเวอร์ตีความข้อความเดียวกันได้ ตัวอย่างเช่น Content-Type บอกชนิดของเนื้อหาที่ส่ง Accept บอกชนิดของเนื้อหาที่ไคลเอนต์ต้องการรับ Authorization ส่งหลักฐานสำหรับการพิสูจน์ตัวตน และ Cookie ส่งข้อมูลสถานะที่เบราว์เซอร์ได้รับจากเซิร์ฟเวอร์ก่อนหน้า เมื่อเราตรวจคำขอในแผงเครือข่าย ให้ดูส่วนหัวควบคู่กับเนื้อหาเสมอ

ส่วนหัว	หน้าที่	ตัวอย่างในร้านค้าแพ
Accept	รูปแบบที่ไคลเอนต์อ่านได้	พรอนต์เอนด์ขอ application/json สำหรับรายการคิว
Content-Type	รูปแบบของเนื้อหาที่ส่ง	ฟอร์มส่งคำสั่งซื้อเป็น application/json
Authorization	หลักฐานพิสูจน์ตัวตน	พนักงานส่งโทเคนเมื่อเปลี่ยนสถานะ คิว
Set-Cookie	เซิร์ฟเวอร์ตั้งค่าคุกกี้ให้เบราว์เซอร์	API ตั้งคุกกี้เซสชันหลังเข้าสู่ระบบ
Cache-Control	เงื่อนไขการเก็บสำเนา	เมนูแคชได้สั้น ๆ ส่วนคิวต้องสดกว่า

MIME type เป็นชื่อมาตรฐานที่บอกชนิดของข้อมูล เช่น text/html, text/css, application/javascript, application/json, image/png, image/svg+xml, video/mp4 และ audio/mpeg เบราวเซอร์ใช้ข้อมูลนี้ตัดสินใจว่าจะอ่าน แสดงผล หรือส่งต่อข้อมูลอย่างไร สำหรับร้านค้าแพ หน้าเมนูอาจเป็น HTML ข้อมูลคำสั่งซื้อเป็น JSON รูปเครื่องดื่มเป็น WebP หรือ JPG ไอคอนเป็น SVG และเสียงแจ้งเตือนเป็น MP3

คุกกี้ช่วยให้ระบบจำสถานะข้ามคำขอหลายครั้งได้ เพราะ HTTP แต่ละคำขอเป็นข้อความแยกกัน บทความความปลอดภัยจะลงรายละเอียดเรื่อง HttpOnly, Secure และ SameSite ในบทนี้ให้จำหลักการก่อนว่า คุกกี้เป็นกลไกส่งข้อมูลกำกับไปกับคำขอ ส่วนการตัดสินใจยังเป็นหน้าที่ของแบคเอนด์

4.5 วิธีการและคุณสมบัติสำคัญ

วิธีการ	เจตนา	ปลอดภัยข้างเคียง/ ทำซ้ำได้	ตัวอย่าง
GET	ขอตัวแทนทรัพยากร (representation)	ปลอดภัยข้างเคียง และทำซ้ำได้	/api/orders/42
POST	ส่งข้อมูลให้ทรัพยากรประมวลผล หรือสร้างสมาชิก	ไม่รับประกัน	/api/orders
PUT	แทนที่ทรัพยากรเป้าหมายด้วย ข้อมูลชุดใหม่	ทำซ้ำได้	/api/orders/42

วิธีการ	เจตนา	ปลอดภัยข้างเคียง/ ทำซ้ำได้	ตัวอย่าง
PATCH	แก้ทรัพยากรบางส่วนตามความ หมายของแพตช์	ขึ้นกับรูปแบบ	/api/orders/42
DELETE	ขอให้ลบความสัมพันธ์กับ ทรัพยากร	ทำซ้ำได้ในเชิง เจตนา	/api/orders/42

ปลอดภัยข้างเคียง (safe) หมายถึงไคลเอนต์มีได้ร้องขอให้เปลี่ยนสถานะ แต่ไม่ได้หมายความว่าผลการประมวลผลจะไม่สร้าง
บันทึกเหตุการณ์หรือไม่มีค่าใช้จ่าย **ทำซ้ำได้ (idempotent)** หมายถึงการส่งคำขอเดียวกันหลายครั้ง ให้ผลตามเจตนาเหมือนส่ง
เพียงครั้งเดียว แม้การตอบกลับแต่ละครั้งอาจแตกต่างกัน

หมายเหตุ: คำสั่งซ้ำและปุ่มสั่งซื้อ

คำขอ POST /api/orders อาจสร้างคำสั่งซื้อใหม่ทุกครั้งที่ส่ง ถ้าเครือข่ายช้าแล้วผู้ใช้กดซ้ำ ระบบอาจมีคำสั่งซื้อ
สองใบ ทีมจึงควรใช้รหัสคำสั่งซื้อซ้ำหรือ idempotency key เพื่อบอกว่า การส่งซ้ำนี้เป็นความพยายามเดียวกัน ไม่ใช่คำสั่ง
ซื้อใหม่ของลูกค้า

4.6 รหัสสถานะเป็นส่วนหนึ่งของข้อตกลง

รหัส	ความหมายในกรณีศึกษา	ข้อสังเกต
200	อ่านหรือแก้ไขข้อมูลสำเร็จและมีเนื้อหาตอบกลับ	ใช้กับ GET/PATCH โดยทั่วไป
201	สร้างทรัพยากรสำเร็จ	ควรส่งรูปแบบและส่วนหัว Location เมื่อเหมาะสม
204	สำเร็จโดยไม่มีเนื้อหาตอบกลับ	ไคลเอนต์ต้องไม่พยายามอ่านเนื้อหาเป็น JSON
400	ไวยากรณ์หรือรูปแบบทั่วไปของคำขอไม่ถูกต้อง	ไม่ควรใช้แทนข้อผิดพลาดทุกชนิด
401	ขาดข้อมูลรับรองที่ใช้พิสูจน์ตัวตนได้	เกี่ยวข้องกับการพิสูจน์ตัวตน
403	ทราบตัวตนแล้วแต่ไม่ได้รับอนุญาต	เกี่ยวข้องกับการกำหนดสิทธิ์
404	ไม่พบหรือไม่เปิดเผยทรัพยากรเป้าหมาย	ต่างจากรายการว่างซึ่งอาจตอบด้วย 200
409	ขัดกับสถานะปัจจุบัน เช่น ค่าที่ต้องไม่ซ้ำเกิดซ้ำ	ไคลเอนต์อาจแก้ไขข้อมูลแล้วส่งใหม่
422	อ่านไวยากรณ์ได้แต่ข้อมูลละเมิดกติกา	เหมาะกับข้อผิดพลาดรายช่องข้อมูล
500	เซิร์ฟเวอร์พบภาวะที่ไม่ได้จัดการ	ห้ามส่งสแต็กเทรซ (stack trace) ให้ไคลเอนต์

4.7 ข้อตกลง JSON และรายละเอียดปัญหา

JSON รองรับออบเจกต์ อาร์เรย์ สตริง ตัวเลข ค่าตรรกะ และ null แต่ไม่มีวินเวลา undefined หรือตัวอย่างของคลาส
โดยตรง จึงต้องกำหนดรูปแบบการแทนค่า เช่น แทนวินเวลาด้วยสตริงตามมาตรฐาน ISO 8601

```

2  "data": {
3    "id": 42,
4    "drink": "Iced Latte",
5    "sweetness": 50,
6    "createdAt": "2026-08-10T09:30:00.000Z"
7  }
8  }

```

การตอบกลับเมื่อเกิดข้อผิดพลาดต้องให้ทั้งโปรแกรมและมนุษย์นำไปใช้ได้ RFC 9457 เสนอรูปแบบรายละเอียดปัญหา (problem details) ที่มีสมาชิกมาตรฐาน เช่น type, title, status, detail และ instance[16]

```

1  {
2    "type": "https://example.edu/problems/validation",
3    "title": "ข้อมูลไม่ผ่านการตรวจสอบ",
4    "status": 422,
5    "errors": {
6      "drink": ["การระบุชื่อก्रेตมมีย่างน้อย 2 อักขระ"]
7    }
8  }

```

4.8 รูปแบบข้อมูลบนเว็บ

รูปแบบข้อมูลเป็นข้อตกลงว่าข้อมูลหนึ่งชุดจะถูกเขียนและอ่านอย่างไร เอกสารอ้างอิงหัวข้อ 6-1 เรียงจาก CSV ไป XML และ JSON ลำดับนี้ช่วยให้เราเห็นว่าความซับซ้อนของข้อมูลต้องสัมพันธ์กับรูปแบบที่เลือก ร้านกาแฟใช้ข้อมูลหลายชนิด: รายการเมนูที่เป็นตาราง คำสั่งซื้อที่มีรายละเอียดซ้อนกัน และเอกสารตั้งค่าหรือข้อมูลแลกเปลี่ยนกับบริการภายนอก

รูปแบบ	เหมาะกับข้อมูล	ตัวอย่างในระบบร้านกาแฟ
CSV	แถวและคอลัมน์ที่โครงสร้างเรียบง่าย	นำเข้าราคาเมนูตั้งต้นจากสเปรดชีต
XML	ข้อมูลลำดับชั้นที่ต้องใช้แท็กและสคีม่าเข้มงวด	เอกสารแลกเปลี่ยนกับระบบเก่าหรือบริการที่กำหนด XML schema
JSON	ข้อมูลโครงสร้างที่ JavaScript อ่านและเขียนได้สะดวก	คำขอสร้างคำสั่งซื้อ รายการคิว และข้อผิดพลาดรายช่อง
JSON Schema	กติกาของ JSON ที่ตรวจสอบได้	ตรวจว่า drinkId, sweetness, customerName มีชนิดและช่วงค่าถูกต้อง

```

sku,name,price_cents
latte-icedลาเต้เย็น,, 6500
thai-tea-icedชาไทยเย็น,, 5500

```



รูปที่ 4.4: เว็บแอปพลิเคชันหนึ่งระบบใช้หลายรูปแบบข้อมูลตามชนิดของเนื้อหา

CSV อ่านง่ายในตารางและนำเข้าข้อมูลตั้งต้นได้เร็ว แต่ข้อมูลซ้อนกัน เช่น ตัวเลือกขนาดแก้ว ระดับความหวาน หรือคำแปลหลายภาษา จะจัดการยากขึ้น XML และ JSON รองรับข้อมูลซ้อนกันได้ดีกว่า XML ใช้แท็กและแอตทริบิวต์คล้าย HTML และตรวจด้วย XML schema ได้ JSON ใช้วงเล็บปีกกา อาร์เรย์ และชื่อเขตข้อมูลที่ JavaScript แปลงเป็นออบเจกต์ได้โดยตรง

```
{
  "drinkId": "latte-iced",
  "customerName": "Mali",
  "options": {
    "size": "medium",
    "sweetness": 50
  }
}
```

การมี JSON ที่ถูกไวยากรณ์ยังไม่พอสำหรับ API ระบบต้องตรวจสอบและกีดกันของข้อมูลด้วย เช่น sweetness ต้องเป็นจำนวนเต็มตั้งแต่ 0 ถึง 100 และ drinkId ต้องอ้างอิงเครื่องดื่มที่ขายจริงในเมนู

4.9 รูปแบบรูปภาพ วิดีโอ และเสียง

ประสบการณ์ของหน้าเว็บขึ้นกับขนาดและชนิดของสื่อด้วย รูปเครื่องดื่มช่วยให้ลูกค้าเลือกเมนูได้เร็วขึ้น ไอคอนช่วยสแกนสถานะวิดีโอสั้นอาจสาธิตวิธีรับแก้วหรือแนะนำเมนูใหม่ เสียงแจ้งเตือนอาจช่วยพนักงานหลังร้านเมื่อมีคำสั่งซื้อใหม่ ทุกรูปแบบต้องสัมพันธ์กับ MIME type, ขนาดไฟล์, คุณภาพ, การเข้าถึงได้ และความเร็วของเครือข่ายในมหาวิทยาลัย

รูปแบบ	จุดแข็ง	ตัวอย่างการใช้
JPG/JPEG	ภาพถ่ายสีจำนวนมาก ขนาดไฟล์ควบคุมได้	ภาพเครื่องดื่มจริงบนเมนู
PNG	ภาพกราฟิกที่ต้องการความคมและความโปร่งใส	ภาพประกอบหรือแผนภาพที่ต้องรักษาขอบชัด
SVG	กราฟิกเวกเตอร์ ปรับขนาดได้โดยไม่เสียความคม	ไอคอนสถานะคิว โลโก้ และแผนภาพที่แก้ด้วย CSS ได้
WebP	ภาพเว็บที่ลดขนาดไฟล์และรองรับความโปร่งใส	ภาพเมนูบนหน้าเว็บที่ต้องโหลดเร็ว
MP4/WebM	วิดีโอในเบราว์เซอร์ผ่าน <code><video></code>	คลิปสั้นสอนวิธีรับเครื่องดื่มหรือสแกน QR
MP3/MP4 Audio	เสียงในเบราว์เซอร์ผ่าน <code><audio></code>	เสียงแจ้งเตือนเมื่อมีคำสั่งซื้อใหม่หลังร้าน

```

1 <picture>
2   <source srcset="images/latte.webp" type="image/webp">
3   
5 </picture>
6
7 <video controls preload="metadata" poster="images/pickup-poster.jpg">
8   <source src="media/pickup-guide.mp4" type="video/mp4">
9   <p เบร้าวเซอรันยั้งไมร่องรบัวดีโอของคุณ้อรับเครอั้งดมี></p>
10 </video>
11
12 <audio controls preload="none">
13   <source src="media/order-ready.mp3" type="audio/mpeg">
14   <p เบร้าวเซอรันยั้งไมร่องรบัเลอั้งแ้งเตอั้น></p>
15 </audio>

```

`<video>` และ `<audio>` ให้เบราว์เซอร์แสดงตัวควบคุมพื้นฐานได้ ค่า `controls` ช่วยให้ผู้ใช้หยุด เล่น หรือปรับเสียงเอง ค่า `preload="metadata"` ช่วยลดการโหลดวิดีโอเต็มก่อนจำเป็น และข้อความสำรองภายในเอลิเมนต์ช่วยให้หน้าเว็บยังสื่อสารได้เมื่อเบราว์เซอร์ไม่รองรับรูปแบบนั้น

4.10 การสื่อสารตามจังหวะของข้อมูล

คำสั่งข้อบางชนิดเหมาะกับ HTTP แบบคำขอและคำตอบ เช่น การโหลดเมนูหรือส่งคำสั่งซื้อใหม่ ข้อมูลบางชนิดเปลี่ยนตามเวลา เช่น หมายเลขคิวที่กำลังเรียกหรือสถานะการชำระเงิน เอกสารหัวข้อ 5-2 แยกวิธีสื่อสารตามจังหวะข้อมูลเป็น polling, long polling, SSE และ WebSocket



รูปที่ 4.5: วิธีสื่อสารเมื่อข้อมูลคิวร้านกาแฟเปลี่ยนแปลงตามเวลา

วิธี	พฤติกรรม	เหมาะกับกรณี
Polling	ไคลเอนต์ถามเซิร์ฟเวอร์เป็นช่วงเวลา	หน้าคิวนาฬิกาเล็กที่ยอมรับความล่าช้าได้
Long polling	คำขอค้างไว้จนมีข้อมูลใหม่หรือหมดเวลา	ลดจำนวนคำขอเมื่อข้อมูลมาไม่ถึง
SSE	เซิร์ฟเวอร์ส่งเหตุการณ์ทางเดียวหลังไคลเอนต์เปิดการเชื่อมต่อ	อัปเดตสถานะคิวหรือประกาศเมนูหมด
WebSocket	เปิดการเชื่อมต่อที่ส่งข้อมูลได้สองทาง	ระบบสนทนา หรือหน้าหลังร้านที่ต้องโต้ตอบสดหลายทิศทาง

สำหรับ Campus Coffee Queue จุดเริ่มที่เหมาะสมคือ polling ช่วงสั้นหรือ SSE เพราะหน้าคิวนส่วนใหญ่รับข้อมูลจากเซิร์ฟเวอร์ทางเดียว WebSocket เหมาะเมื่อมีการโต้ตอบสดหลายฝ่าย เช่น พนักงานหลายคนแก้คิวพร้อมกัน หรือระบบต้องส่งเหตุการณ์กลับไปกลับมาอย่างต่อเนื่อง

4.11 เฟตช์เอพีไอ: พรอมิสสำเร็จไม่ได้แปลว่าสถานะ HTTP สำเร็จ

```

1 async function listOrders({ signal } = {}) {
2   const response = await fetch("/api/orders", {
3     headers: { Accept: "application/json" },
4     signal
5   });
6
7   if (!response.ok) {
8     const problem = await readProblem(response);
9     throw new HttpError(response.status, problem);
10  }
11
12  return response.json();
13 }
  
```

พรอมิสของ Fetch จะถูกปฏิเสธเมื่อคำขอล้มเหลวในระดับเครือข่ายหรือถูกยกเลิก แต่การตอบกลับ 404 หรือ 500 ยังคงเป็นการตอบกลับ HTTP ที่สำเร็จในระดับการขนส่ง จึงต้องตรวจสอบ `response.ok` หรือ `status` เอง เนื้อหาการตอบกลับเป็นสายข้อมูล (stream) และโดยทั่วไปอ่านได้เพียงครั้งเดียว[13]

```

1 async function readProblem(response) {
2   const type = response.headers.get("content-type") ?? "";
3   if (type.includes("application/json")) {
  
```

```

4   return response.json();
5   }
6   return { title: await response.text() };
7 }

```

4.12 สถานะของส่วนติดต่อผู้ใช้ที่ครบถ้วน

การออกแบบเฉพาะสถานะสำเร็จทำให้ผู้ใช้ไม่ทราบว่าคำขอกำลังทำงาน ไม่พบข้อมูล หรือเกิดข้อผิดพลาด มุมมองของทรัพยากรหนึ่งรายการควรพิจารณาอย่างน้อย

1. ยังไม่เริ่ม (idle): ยังไม่ได้ส่งคำขอ
2. กำลังโหลด (loading): กำลังรอและควรป้องกันการกระทำซ้ำเมื่อจำเป็น
3. สำเร็จและมีข้อมูล: แสดงข้อมูลที่ได้รับ
4. สำเร็จแต่ไม่มีข้อมูล: คำขอสำเร็จแต่ชุดข้อมูลว่าง
5. ผิดพลาด: แสดงข้อความที่ช่วยให้ผู้ใช้ดำเนินการต่อได้
6. ล้มสมัยหรือถูกยกเลิก: เป็นผลเก่าหรือคำขอที่ไม่ควรเปลี่ยนหน้าปัจจุบัน

```

1 const controller = new AbortController();
2
3 try {
4   renderLoading();
5   const result = await listOrders({ signal: controller.signal });
6   result.data.length === 0
7     ? renderEmpty()
8     : renderOrders(result.data);
9 } catch (error) {
10   if (error.name !== "AbortError") renderError(error);
11 }

```

4.13 นโยบายแหล่งกำเนิดเดียวกันและคอร์ส

แหล่งกำเนิด (origin) ประกอบด้วยแบบแผน (scheme) ชื่อโฮสต์ และพอร์ต เบราร์เซอร์จำกัดสคริปต์มิให้เข้าถึงการตอบกลับจากต่างแหล่งกำเนิดโดยไม่มีนโยบายอนุญาต การแบ่งปันทรัพยากรข้ามแหล่งกำเนิด (Cross-Origin Resource Sharing: CORS) เป็นกลไกส่วนหัวที่เซิร์ฟเวอร์ใช้ประกาศสิทธิ์ให้เบราร์เซอร์ มีใช้ระบบพิสูจน์ตัวตน และมีได้ป้องกันเซิร์ฟเวอร์อื่นหรือ curl จากการส่งคำขอ

ข้อควรระวัง: อย่าแก้ CORS ด้วยการเปิดทั้งหมดโดยไม่วิเคราะห์

ต้องระบุแหล่งกำเนิดของฟรอนต์เอนด์ที่จำเป็น รวมทั้งวิธีการและส่วนหัวที่ใช้ หากมีข้อมูลรับรอง ห้ามใช้สัญลักษณ์ครอบจักรวาลเป็นแหล่งกำเนิด และทุกเส้นทางยังต้องตรวจการพิสูจน์ตัวตนกับการกำหนดสิทธิ์ตามปกติ

4.14 ใช้ curl และแผงเครือข่ายเป็นเครื่องมือแยกชั้นปัญหา

```

1 curl -i http://localhost:3000/api/orders
2
3 curl -i -X POST http://localhost:3000/api/orders \
4   -H "Content-Type: application/json" \
5   -d '{"drink": "Iced Latte", "sweetness": 50, "customerName": "Mali"}'
```

–i แสดงส่วนหัวของการตอบกลับ การส่งคำขอโดยไม่ผ่านพรอนต์เอนต์ ช่วยวินิจฉัยว่าข้อผิดพลาดเกิดใน API หรือในโค้ดของเบราว์เซอร์ แผงเครือข่ายให้ข้อมูลเพิ่มเติม เช่น เวลา จุดเริ่มคำขอ ข้อมูลที่ส่ง และตัวอย่างการตอบกลับ หลักฐานทั้งสองควรสอดคล้องกันก่อนสรุปสาเหตุ

4.15 ปฏิบัติการ: ไคลเอนต์ของ API

1. สร้างหน้ารายการที่เรียก API จำลองหรือ API ของปฏิบัติการด้วย GET
2. แสดงสถานะกำลังโหลด ไม่มีข้อมูล สำเร็จ และผิดพลาดให้แยกจากกัน
3. สร้างคำขอ POST จากแบบฟอร์มและจัดการสถานะ 201 กับ 422
4. ยกเลิกคำขอเมื่อเริ่มค้นหาใหม่หรือเลิกใช้คอมพิวเตอร์
5. ส่งคำขอเดียวกันด้วย curl และบันทึกวิธีการ URL เนื้อหาคำขอ สถานะ ชนิดเนื้อหา และเนื้อหาการตอบกลับ
6. สร้างตารางกรณีทดสอบสำหรับ 200, 201, 204, 404, 422, 500 และกรณีเครือข่ายล้มเหลว

4.16 เกณฑ์ประเมินและคำถามทบทวน

มิติ	เกณฑ์	น้ำหนัก
ความหมายของ HTTP	วิธีการ เส้นทาง และสถานะสอดคล้องกับเจตนา	25%
ข้อตกลงข้อมูล	JSON สำหรับกรณีสำเร็จและผิดพลาดมีโครงสร้างคงที่และอธิบายได้	20%
การจัดการ Fetch	แยกสถานะ HTTP เครือข่ายล้มเหลว ข้อมูลว่าง และการยกเลิก	25%
การวินิจฉัย	หลักฐานจาก curl และแผงเครือข่ายทำให้เกิดปัญหาซ้ำได้	20%
การเข้าถึงได้	ข้อความสถานะได้รับการประกาศและการกระทำใช้คีย์บอร์ดได้	10%

กิจกรรมตรวจสอบความเข้าใจ: คำถามทบทวน

1. ตัวแทนทรัพยากรต่างจากตัวทรัพยากรอย่างไร
2. ความปลอดภัยข้างเคียงและการทำซ้ำได้เป็นคุณสมบัติเดียวกันหรือไม่
3. เหตุใด Fetch จึงไม่ปฏิเสธพอร์มิลโดยอัตโนมัติเมื่อได้รับสถานะ 404
4. สถานะ 404 ต่างจากชุดข้อมูลว่างอย่างไร

5. CORS ป้องกันสิ่งใด และเหตุใดจึงใช้แทนการกำหนดสิทธิ์ไม่ได้
6. ออกแบบหลักฐานเพื่อแยกปัญหาของฟรอนต์เอนด์ออกจากปัญหาของ API

4.17 สรุปสาระสำคัญ

HTTP เป็นข้อตกลงเชิงความหมายระหว่างไคลเอนต์กับเซิร์ฟเวอร์ JSON เป็นรูปแบบสำหรับแลกเปลี่ยนข้อมูล มีใช้แบบจำลองข้อมูลทั้งหมดของระบบ Fetch ต้องจัดการสถานะการตอบกลับ การอ่านเนื้อหา การยกเลิก และสถานะของส่วนติดต่อผู้ใช้อย่างชัดเจน ส่วน `curl` กับแผงเครือข่าย ทำให้การสื่อสารที่มองไม่เห็นกลายเป็นหลักฐานตรวจสอบได้ บทถัดไปนำข้อตกลงนี้เข้าสู่สถาปัตยกรรมคอมโพเนนต์ของ React